

Smart Dynamic Pricing Engine for E-Commerce

An AI-powered **dynamic pricing engine** continuously ingests market data (competitor prices, demand trends, inventory levels) to adjust product prices in real time. By analyzing competitor prices on platforms like Amazon, Walmart, eBay, etc., alongside our sales history, it recommends optimal price points that maximize revenue and profit margins ¹ ². Studies show that AI-driven pricing can boost gross profit by 5–22% on average and reduce excess inventory by ~30% ³ ⁴. For example, Amazon adjusts prices millions of times per day, yielding ~25% higher revenue ³ ⁵. Our system leverages these insights to automatically undercut or align with competitors' pricing while considering demand forecasts and stock constraints.

Target Marketplace Platforms

- **Amazon** – via Seller Central API or scraping. Win the Buy Box by reacting to other Amazon sellers ⁶.
- **eBay, Walmart, Alibaba** – integrate via their public APIs or data feeds.
- **Shopify/Magento** – for direct-to-consumer stores, connect through their APIs.
- **Google Shopping, Target, Etsy, etc.** – additional channels as needed.

We monitor **multiple marketplaces and channels** to get a broad competitive view ⁷. This cross-platform data lets us dynamically reprice products to stay competitive in each market.

Data Inputs & Mock Data Strategy

- **Competitor Pricing & Stock:** Continuously scrape or use APIs to collect rivals' prices and inventory on marketplaces ². (Initial prototyping will use mock or historical price data; later switch to live web-scraping.)
- **Demand & Sales Trends:** Use internal sales logs (web analytics, point-of-sale data) to gauge historical demand. This data fuels our forecasting models.
- **Inventory Levels:** Track current stock for each SKU to avoid stockouts or overstock.

For the hackathon prototype, we'll generate or use open e-commerce datasets (e.g. Kaggle sales data) as **mock data**. This allows us to demonstrate forecasting and pricing logic without live web data ⁸. Eventually, the pipeline will ingest real-time data via AWS Kinesis or batch jobs.



Competitor price monitoring is core to this system. We continuously gather pricing and availability data from major marketplaces ². By tracking rivals' pricing strategies, our engine dynamically adjusts our prices to capture market share and margin ² ⁵. For example, if a competitor runs low on stock, our system can raise prices to increase profit. Likewise, if demand dips, it may lower prices or run promotions.

Time-Series Demand Forecasting

We implement **time-series forecasting** to predict future demand for each product ⁹. Using AWS tools (Amazon Forecast, SageMaker), we train models (ARIMA, Prophet, RNN/DeepAR, etc.) on historical sales data. Accurate demand forecasts let us set prices proactively: if we predict a surge (e.g. before a holiday), we can raise prices to maximize revenue; if demand is weak or inventory is piling up, we lower prices to clear stock ¹ ⁹.

Our workflow might use **Amazon Forecast**, a managed ML service specialized for business time-series data ⁹, or SageMaker notebooks with custom models. Forecast accuracy is tracked by metrics like MAPE/RMSE. The forecast horizon can be hourly or daily, enabling dynamic repricing every few minutes if needed.

Pricing Optimization and Model

The core AI model combines **competitive intelligence** and **demand forecasts** to calculate optimal prices. Key components:

- **Price Elasticity Modeling:** Estimate how demand changes with price. If a small price drop triggers a large sales increase (high elasticity), the model may lower prices to boost volume.
- **Profit-Driven Optimization:** Solve for price that maximizes profit = (price – cost) × expected sales, subject to business rules (e.g. minimum margin, MAP policies).
- **Rule-Based Constraints:** Enforce bounds (e.g. never below cost, or \$X above lowest competitor) to meet business rules.
- **Machine Learning Policy:** Optionally use reinforcement learning or multi-armed bandit algorithms to automatically explore/prioritize pricing strategies.

The output is a **recommended price per product**. For example, rules can include “price \$5 below the highest competitor” or “increase price 10% if forecasted demand exceeds threshold” ⁶ ⁵. The system can simulate candidate prices against historical data to validate the revenue impact before applying them.

System Architecture and AWS Integration

Our solution is built on **AWS cloud services** for scalability and real-time performance. The architecture is agent-based and serverless ¹⁰ ¹¹:

- **Data Ingest & Storage:** Real-time competitor prices and sales logs stream into **AWS Kinesis** or **AWS IoT**, then land in **Amazon S3** or **DynamoDB**. DynamoDB holds up-to-date pricing data for instant access.
- **Demand Forecast Agent:** Periodically (or on new data) triggers **Amazon SageMaker** or **Amazon Forecast** jobs to produce demand forecasts ¹² ⁹. SageMaker can train/deploy LSTM/DeepAR models on historical sales.
- **Pricing Agent:** A compute component (AWS Lambda or ECS) combines the latest competitor data and demand forecast to compute price recommendations. This agent can leverage **Amazon Bedrock** (e.g., Anthropic Claude) to analyze contextual data if needed ¹³, or run custom code for numeric optimization.
- **Workflow Orchestration:** **AWS Step Functions** coordinates the end-to-end workflow ¹⁴: triggering data fetchers, forecasting jobs, and pricing updates in sequence. Step Functions ensure reliable, stateful execution and handle retries/errors.
- **Execution/Updates:** Approved price changes are pushed to sales channels via **Amazon API Gateway** or Lambda functions that call each marketplace’s pricing API (or simulate such API calls in the demo) ¹⁵ ¹¹.
- **Monitoring & Alerts:** **AWS CloudWatch** and **SNS** track KPIs (forecast accuracy, price update latency) and alert stakeholders on anomalies.

Key AWS services used: **Amazon SageMaker** (ML model training/deployment), **Amazon Forecast** (time-series forecasting) ⁹, **AWS Lambda** (serverless compute), **DynamoDB** (fast scalable data store), **Step Functions** (orchestration), **API Gateway** (front-end), and **SNS/SQS** (notifications/queuing). This cloud-native, microservice design ensures the system can scale to thousands of products and price updates per minute ¹¹.

Mock Data and Demonstration

For the hackathon demo, we use **mock/synthesized data** to represent competitor prices and sales. Example steps:

1. **Generate Synthetic Dataset:** Create sample SKUs with historical sales (daily units sold) and cost, along with competitor prices (which vary daily) and inventory.
2. **Preprocessing:** Load mock data into S3 or DynamoDB. Use Python scripts or AWS Glue to clean and normalize.
3. **Forecasting Demo:** Run Amazon Forecast or a SageMaker notebook on the sample sales data to show demand curves and predicted future sales.
4. **Pricing Simulation:** Apply the pricing algorithm on this data stream to output new recommended prices.

This approach demonstrates each pipeline stage (data ingestion, forecasting, pricing) in a controlled setting. Once validated, it easily swaps in real-time webscraped data without changing the core logic.

Business Value and Innovation

Implementing this dynamic pricing engine delivers clear **business benefits**:

- **Revenue & Profit Lift:** By always staying competitively priced, retailers can see significant revenue increases. For instance, market reports cite ~12–25% revenue lifts for large e-commerce players using dynamic pricing ³ ¹⁶. AI-driven repricing typically boosts gross profit by 5–22% ⁴.
- **Inventory Efficiency:** Smarter pricing reduces overstock and waste. A documented case showed AI pricing cuts inventory waste by ~27% ¹⁶. Lowering prices on slow-moving stock or raising them on scarce items improves turnover.
- **Customer Competitiveness:** Automated repricing means our store is always competitive on price, improving Buy Box win rates and market share without manual effort.
- **Operational Speed:** Real-time automation accelerates response to flash sales or competitor moves. Managers avoid tedious manual updates and can focus on strategy.

Innovation & Technical Depth: This solution combines **cutting-edge AI/ML** with a scalable cloud architecture. It uses advanced forecasting models (e.g. LSTM, Prophet) and potentially generative AI (LLMs via Bedrock) to analyze complex pricing factors. The agentic design (akin to AWS hackathon examples ¹⁷ ¹¹) allows modular development and easy experimentation (e.g. swapping algorithms). By leveraging a wide array of AWS services (Forecast, SageMaker, Step Functions, etc.), the project demonstrates deep technical integration and industry best practices.

Key performance metrics include *forecast accuracy* (MAPE/RMSE on demand), *pricing recommendation lift* (increase in conversion or revenue), and *system latency* (time from data change to new price). These KPIs ensure the solution not only works technically but also delivers quantifiable business impact ¹⁸ ¹.

Conclusion and Next Steps

This **Smart Dynamic Pricing Engine** meets all hackathon requirements: it targets multiple e-commerce marketplaces, uses time-series forecasting, emphasizes innovation, and is built on AWS tools. Next steps include refining models with real data, integrating live competitor scraping, and implementing a user interface/dashboard. For now, our AWS-based prototype with mock data convincingly shows how AI can revolutionize e-commerce pricing strategy, driving higher profits and agility in competitive markets ¹ ¹⁹.

Sources: Industry analyses and case studies on AI dynamic pricing ¹ ⁴ ² ¹⁹; AWS documentation on forecasting and cloud architecture ⁹ ¹¹.

¹ ³ ⁴ ⁵ Dynamic Pricing ROI: Case Studies from eCommerce | Onramp Funds

<https://www.onrampfunds.com/resources/dynamic-pricing-roi-case-studies-from-ecommerce>

² ⁷ How Competitor Price Monitoring Boosts E-Commerce Success

<https://growbydata.com/competitor-price-monitoring-for-ecommerce-success/>

6 Complete Guide To Amazon Dynamic Pricing Strategy in 2025

<https://myamazonguy.com/amazon-product-launch/amazon-dynamic-pricing-strategy/>

8 16 Dynamic Pricing Engine For E | PDF

<https://www.scribd.com/document/960505148/Dynamic-Pricing-Engine-for-E>

9 How to improve your forecasts and deliver better results in 2023 | AWS for Industries

<https://aws.amazon.com/blogs/industries/how-to-improve-your-forecasts-and-deliver-better-results-in-2023/>

10 11 12 13 14 15 17 18 19 Revolutionizing Retail: Our AI Agentic Approach to Dynamic Pricing & Promotions at AWS Hackathon | by Sameer Choudhary | Medium

<https://medium.com/@samhaldia123/revolutionizing-retail-our-ai-agentic-approach-to-dynamic-pricing-promotions-at-aws-hackathon-4af598414830>